

Lec 37 Practical Exercise - 2

Lec 37 Practical Exercise - 2

Lecture	37 / 63 + intro
Duration	40:05
Captions	English (auto-generated)
Words	5,815
Video	nOL6et_IQRQ
Playlist	Fundamentals of Generative AI and LLMs

Auto-generated YouTube captions, lightly cleaned (removed [music] tags, grouped into paragraphs). These are **verbatim lecture captions**, not edited notes.

Transcript

[00:00] Hello all. Welcome to the hands-on series on fundamentals of generative AI and large language models theory and practice. In today's session, we'll be learning the implementation of deep convolutional generative adversarial networks, in short called as DCGANs. In the previous session, we have seen the architecture of vanilla GAN, which is basically made up of a set of fully connected layers. But the drawback in case of vanilla architecture is the use of FC layers or the fully connected layers, which convert the 2D dimensions of images into a flattened vector or a 1D vector. So with this, the spatial information of the images will be lost. But in case of DCGANs, we make use of the convolutional

[00:50] layers, which tries to preserve the spatial information of images. Also, it considers the hierarchical of features, that is, it considers the low-level features comprising of edges and corners. It tries to maintain or retain the mid-level of features, which includes the textures or the textures or the shapes, and the high-level features, which includes the object parts. Also, the memory requirement in case of DCGANs will be slightly less when compared to the vanilla GANs because of the use of less or fewer parameters. So let us now see the implementation of DCGANs.

[01:35] So the initial portion is importing the necessary libraries. So we import the TensorFlow as TF, which is a deep learning framework used to develop the neural networks. From TensorFlow Keras, we import the layers in order to have sequential layers like input layers, the hidden layers, and the output layers. We import the numerical Python as NP, which is required for dealing with numbers or arrays, and importing the matplotlib.pyplot as PLT, required for data visualization.

[02:06] So, next coming to loading the data set. So, even to have a comparative study between the DCGAN and the vanilla GAN, we make use of the same fashion MNIST data set, which is related to some clothing and accessories, which comprises of 10 different classes of data, varying from 0 to 9 class. Class 0 comprising t-shirts and top, class 1 corresponding to trouser, 2 as pullovers. It has a sets of dress, coat, sandal, shirt, sneakers, bags, and ankle boot. So, comprising of 10 different classes.

[02:44] So, now coming to the loading of the data set and initial pre-processing. So, fashion MNIST data set to be loaded into a variable called as X_train. So, in in case of GANs, since it is a unsupervised learning, we do not need the labels. So, that's why we don't have a variable or parameter called as Y_train and Y_test. And also, the GANs make use of two neural networks. One is the generator and the other is the discriminator. So, there is no point in having the test data set here, because the complete testing is done on the images or the fake images that are generated by the generator. So, that's why we need to consider only the X_train.

[03:29] And then coming to normalizing the data in the range -1 to 1. Why -1 to 1 is because we are making use of the tan h activation function. Tan h activation function gives the values in the range -1 to 1. That's why we have the setup like converting the X_train into floating data type. And then we have this minus symbol and 127.5 because half of 255 or we have the image pixel values ranging from 0 to 255. So, to have the mid value, we have this subtraction.

[04:04] 127.5 / 127.5. We have already seen this normalization. How do we normalize in the range -1 to 1 in case of vanilla GAN? And then we add the channel dimension. So, we need make use of this function called as expand dimensions from the NumPy package in order to have the dimensions, in order to have a 2D architecture. That's why when we display or when we print this X_train shape, we have 60,000 samples in the training set of the size 28 cross 28 cross 1. So, 28 pixels in width, 28 pixels in height, and one because of one channel due to a grayscale image that we are using in the Fashion MNIST data set. So, with the help of expand dimension functions, we are able to get this kind of dimensions.

[04:59] And what we do is this dimensions, we try to apply on the X_train. So, that's why this corresponds to the shape of the training data set. Now, coming to defining the pipeline required to develop this GAN architecture. So, initially we have this number of training samples as 60,000. So, we can call it as a buffer size. Let us have

the training in the form of batches. So, let the batch size be 128. So, we have this from the TensorFlow data, we take this data set, and we have slices of `X_train` in the form of batches. We will try to shuffle the buffer size. We will always try to shuffle the training samples because let us not have index 0 to 127, then 128 to 255 in the same order. So, in order to avoid this memorizing of the

[05:52] training samples, let us have the shuffling of samples. So, this is called as shuffling of the data samples. So, this is all about the pipeline that we set in order to have the data extraction from the training samples. Now, coming to building of the deep convolutional generative adversarial network. In this, the generator network. What exactly does a generator takes as input and what is the output from the generator? So, let us see that in this particular code cell.

[06:24] So, as mentioned in the previous session, the GAN network the GAN network makes use of two neural networks, the generator and the discriminator. The generator makes use of the latent dimensions or the latent vector created from a random noise. That's why we need to define this latent dimension is equal to 100. So, we build a generator and this is sequential stacking of the layers one after the other. We add the layers, which is the dense layer or the hidden layers of the dimension 7 cross 7 cross 256.

[06:57] So, 256 neurons are required or we can say that we are generating 256 feature maps of the size 7 cross 7. Now, why to have the 7 cross 7? We can always have 14 cross 14. We can also have 21 cross 21 or 28 cross 28. Basically, why 7 cross 7 is to have to get the original pixel size, that is 28 cross 28, we should have a multiples of 7. That's why we take from the minimum possible dimension, that is 7 cross 7. Totally, we generate 256 feature maps after convolution of the size 7 cross 7.

[07:36] Now, we try to avoid this bias term here. That's why we have this argument use bias is equal to false. So, generally we have this number of parameter calculation, which is varying in terms of the input neurons, the hidden neurons, and the biases associated with it. But, in in case of DC GANs, we try to avoid this because we make use of the batch convolution, we make use of certain activation function. So, when we use the batch normalization, by default we have the scaling parameter as well as the shifting parameter. So, to avoid the shifting of the data twice, we try to avoid this bias term.

[08:16] So, our dense layer completely dependent on the feature maps here and the input shape, which is of the shape latent dimension, which is equal to 100. So, then we have the layers batch normalization is being added and then we have the ReLU activation function, which acts some non-linearity to the data. Finally, we reshape the data into the size $7 * 7 * 256$. And then we have this convolutional transpose so that we perform the up sampling of the data in order to get the sharp features present in

the data.

[08:53] That's why we have this transpose here. So, for that we we take 128 neurons and kernel size is 5×5 , stride is equal to 1. We try to use some padding so that's why we say padding is equal to same so that we throughout the borders of the image we have all the paddings done, that is zero padding done. And bias is once again false. We have next set of batch normalization followed by the activation function ReLU. And once again, we have this convolutional transpose layer with 64 neurons, 5×5 as the kernel size, strides being 2×2 , and the padding will be the same. Once again, we use this paddings to maintain the borders of the image and biases false. So, likewise, we have combination of convolutional layers, the batch

[09:42] normalization process, and then the ReLU activation function until we reach the number one here. So, the final one neuron uh with 5 cross 5 kernel size, strides as 2 cross 2, padding will be same, and the activation function is tan h. So, after this convolution, we get all the values in the range -1 to 1. So, let us see how exactly this convolution works, batch normalization, as well as the ReLU function works. So, first coming to the convolution part.

[10:27] For simplicity, let me take a 5 cross 5 image. Since this is a grayscale image, values will range in from 0 to 255. So, maybe this is 1 1 0. 2 10 So, 0 corresponds to black pixels, and 255 corresponds to the brighter side of the pixels. So, we have this 5 cross 5 image. Now, if we take a kernel of size 3,3, that is 3 cross 3. Kernel will try to extract certain features, maybe detection of edges, detection of corners, or detection detection of lines or points in the image. So, we can use several set of uh operators or filters. Maybe the private operator, the Sobel operator, or maybe the Canny operator based on the requirement. Say, for example, if I'm using the private operator, and uh the values will be

[11:49] -1 -1 -1 0 0 0 1 1 1. This is basically the vertical operator. When I try to superimpose this on the given image, so what happens the middle zero will be put on to this value. So, each of the values are multiplied. So, -1 into 1 0 into 1 1 into 0 -1 into 2 0 into 10 this 1 into 1 -1 into 3 0 into 1 1 into 2. So, whatever value we get after this convolution, we try to replace in place of 10. Similarly, next time the same convolution, I mean, the same kernel is put on this value one.

[12:37] In the third iteration, it is put on zero. Likewise, we try to get values only on these places. So, from 5 cross 5 image, now after down sampling, we get only 3 cross 3. So, what has happened to the edges? The information on the edges will be lost. So, in order to avoid that, we try to pad some zeros on the corners or edges of the image. So, by doing that by doing that, what happens is we can put the center value zero on this one, try to have the multiplication.

[13:27] Now, this becomes our 3 cross 3 image. Next value, we try to put the zero on the next value, that is one. So, now this becomes our 3 cross 3 image. So, with this

the border values the boundaries will also be maintained. So, that is a use of padding. So, whatever operation we perform, that is multiplication of each of the pixel values and a sum of them, that forms our convolution. So, whatever output that we get from this operation is called as the feature map. So, earlier we have seen that we get 256 such feature maps of the size 7 cross 7.

[14:10] Seven pixels in width and seven pixels in the height. That's why we get 7 cross 7 and 256 different feature maps. Now, coming to batch normalization. So, batch normalization in order to have a normalized values or values in a particular range. So, if we take the original value as X , then the normalized value X_{norm} will be X minus the original value minus the mean value divided by the variance plus epsilon. So, this epsilon is a small numerical value in order to have the numerical stability. That means basically to avoid the division by zero, we can make use of this small value. Otherwise, the importance of this epsilon is very much negligible. So, it is basically $X - \mu$ divided by the standard deviation.

[15:03] So, square root of sigma square will give us sigma. Sigma is nothing but the standard deviation. So, with this we will be able to check how much the original value has shifted from the mean value. Ideally, whenever we take the Gaussian distribution of data, we have the mean value as zero and standard deviation as one. So, that's why we have this bell curve kind of data. And we always try to avoid the larger values both on the negative side as well as the positive side or maybe towards the high or lower side as well as the higher side.

[15:36] So, with this normalized formula, we are getting a values in a range that that varies from one to minus one to one standard deviation or maybe minus two to two standard deviation or minus three to three standard deviation based on how much percentage of of the data we want within that bell curve. Now, with this normalized value, we get the output value which is $\gamma X_{norm} + \beta$. So, γ refers to the learnable scalar and β refers to the learnable shift.

[16:22] Since we already consider this β and try to shift this normalized value, there is once again no necessity of adding the bias. Usually, what happens is while calculating the parameters, what we do is number of input neurons into hidden neurons plus the bias. This is what we add. So, this bias or the shift is already got with the help of this β . That's why in this DC again, we try to make it zero or we try to completely avoid this bias.

[16:54] So, whatever parameter calculation we do, it is based on number of input shape or the input neurons into the hidden neurons. Now, after this batch normalization, we have the leaky ReLU or the ReLU. We have the ReLU activation function, which is basically $\max(0, x)$. Whenever x is negative, the maximum value is 0 the negative value. So, negative value will give the value as 0. Whenever x is positive, 0 the positive value, it is the same value or the positive value. So, with this we try to

add some non-linearity to the input data. So, these are the set of operations we try to do in case of convolutional layers or DCGANs.

[17:45] Now, one more thing is use of stride. So, in the example what we have taken, 5 cross 5 example what we have taken. If I'm trying to take this window, this particular window, okay? Do some convolution on the center pixel, get a new value. And in the next iteration, if I take this particular window, and I'm trying to exclude this first column. I'm moving only by one unit. So, in this case, stride will be equal to one.

[18:23] But, what we try to do is we are in our implementation, we are making the stride value is equal to two. So, in case of stride value is equal to two, if this is the first window that we take, stride value is equal to two is two jumps. So, my center pixel will be here, and that's why the new window is this. The new window is after two shifts. So, these are some of the operations that we try to have in this generator network.

[19:02] So, finally we generate the summary of the generator. So, we have the first dense layer with the output shape as 12,544, and number of parameters are 1,254,400. So, how do we get this 1,254,400 is Let us go to the first layer. To the first layer, the input shape is 100 100 into the hidden layer, which makes use of 7 cross 7, that is 49 into 256. So, basically, it is 100 into 7 into 7 into 256. In the previous case, where bias was used, we would once again add 7 into 7 into 256. So, in our in this case of DC GAN, we avoid this bias. So, it is basically input neurons or input shape into the hidden neurons.

[20:04] So, how is the calculation? Input neurons, we had 100 latent dimensions into 7 into 7 into 256. This is going to give us the number of trainable parameters as 1,254,400. So, with the same concept, we can do the number of we can do the calculation of trainable parameters for this entire summary. So, when we sum up all these trainable parameters, we get the total number of parameters as 2,330,944 different parameters. So, for such a huge architecture, the number of total you know, parameters are slightly less when compared to the vanilla GAN architecture. So, that's why in the beginning we already mentioned that having the convolution layer over the flattened layer or over the FC layer is slightly

[20:58] better in terms of the parameters, because very fewer parameters we are able we we need to calculate, and that's why the training is a little faster, the memory requirement is also less. Now coming to the discriminator network under the DCGAN, how do we build this discriminator? Once again, we have the sequential layer, we add the convolutional layers, and the number of neurons will be exactly opposite to that of the generator. So, we started with the 256 and ended with 64 and finally one. So, here we start with 64 and then we go to 128 and finally one.

[21:38] So, which is able to classify. With one neuron, it is able to act as a classifier

and classify whether we have the fake images or the real images. So, let us see the architecture. We have 64 neurons and the kernel size is 5 cross 5 with the strides being 2 {comma} 2. So, strides both on the width as well as the height. That means we have to have two jumps both on the width as well as the height when we consider the convolution process. And padding is same, which means that we need to uh pad zeros on the boundaries of the image. And the input data shape is 28 cross 28 cross 1.

[22:16] So, referring to width, the height, and one referring to the channel because it is a grayscale image. Now we have this leaky ReLU of the value 0.2. So, how does this leaky ReLU work? This will consider the maximum value of x {comma} a into x . In our case, a value is equal to 0.2. So, whenever we have a negative value, what happens is if say x is -5, then it is -5 {comma} 0.2 times -5. This will be maximum of -5 {comma} -1, which happens to be minus one. So, we still we still keep the neuron active. What if x is equal to five? So, maximum of five {comma} point two into five, which is maximum of five {comma} one, which is equal to five. So, even in this case of positive value, we still keep the neuron active.

[23:24] So, that's the benefit of having the leaky ReLU when compared to the ReLU. And we also add some dropout layers so that we can drop the functioning of a certain neuron so that they one neuron will not try to memorize the values from the previous neuron. So, let the dropout percentage be 30%. 30% of the neurons will be inactive during training. And then we have the next convolution layers with the 128 neurons and kernel sizes five cross five, with the strides being two cross two and padding on the edges. Uh and then we have the reverse process, that is leaky ReLU, once again the dropout layers, and finally we have the flattened layer, which will give us the 1D array. So, we have the one-dimensional array

[24:11] because there are 10 different values or 10 different classes we have an array of 10 different values, and finally we have a dense layer of one neuron. That one neuron will say whether it is a fake image or a real image. So, it basically acts as a binary classifier. So, we have built this discriminator. So, when we have the summary of discriminator, we have the first convolutional layer, so output shape is 14 cross 14 cross 64.

[24:40] So, the calculation will remain the same way. It is the input shape 28 cross 28 cross one into the hidden neurons, which is 64, plus the hidden neurons. We have not made the bias as false here, plus 64. So, which will give us 1,664. So, likewise, when we can calculate the parameter under each of the training. So, convolution, second convolutional layer, and finally the dense layer. We have the total number of parameters, which is going up to 212,865.

[25:24] So, once we have discussed about the generator and the discriminator, now we have to define the loss function, the adversarial loss function. So, loss function

could be binary cross entropy loss function, which takes into account both the discriminator as well as the generator loss. So, what is this cross entropy? The binary cross entropy. From logits is equal to true means So, what is this logits is equal to true? So, logits is equal to true means we get at the flattened layer a vector, okay, a 1D vector of 10 different values. Say, 0.03, 0.1, 0.15, 0.002.

[26:10] So, likewise, 10 different values. So, these are nothing but the raw scores. These are no more the probability, but this has to be converted into a probability value. So, that's why we have this logits is equal to true. If you don't have this, then this would be considered as a probability value, but when we sum up these raw score, it will not sum up to one. That's why these are considered as just the raw score, which has to be later converted into the probability. Usually, we make use of the sigmoid activation function, $\sigma(X)$ will be equal to $1 / (1 + e^{-X})$.

[26:49] Each of these values will have to be substituted here and whatever we get, that becomes our probability value. So, based on this probability, if the probability is greater than or equal to 0.5, we may consider it as a real image. If the probability is less than 0.5, we may consider it as a fake image based on whatever labeling we have given. So, what we try to do is if probability is greater than or equal to 0.5, we consider class as one. So, what is class one? It is the real image which was present in our data set. If probability is less than 0.5, we may consider it as class zero. So, let us label this class zero as the fake images, that is given by the generator.

[27:41] from Fashion MNIST. So, discriminator loss based on the real output and the fake output. So, real output are the real labels or the real images present in the Fashion MNIST data set. Fake output are the one which is given by the generator. So, what is the real loss? It is the cross entropy of the real output over the real output. That is, whatever labels we have for the real output and whatever the real output looks like. So, uh as mentioned in the previous vanilla GAN, we have to give a certain labels either by using ones or zeros. For the real output, let us have a set of ones.

[28:31] That's why I have mentioned class as one. Now, coming to the fake ones. What is this fake loss? It is a cross entropy. How do we get this entropy or the difference in the difference in the prediction? That is by considering the zeros. So, class zero or label zero, which is the fake output. So, now written both the real loss as well as the fake loss. So, generator loss is based on the fake output. So, cross entropy of ones like fake output based on the fake output.

[29:05] So, ones like is once again the labels, which is given like ones, a set of ones or a array of ones. So, this will give us the adversarial loss functions. Now, coming to configuration of the optimizers both for the generator and the discriminator. So, we can make use of any optimizer. In our case, we are making use of the Adam optimizer, which considers both the mean estimate and the variance estimate. And it is based on certain scalar, that is beta one and beta two. Uh we usually make use of beta

values as 0.5, so that we have a neutral balance between the past values or the past images or the real images or maybe the fake images, the fake images generated by the generator. So, if we have beta value too less or too high,

[29:58] the number of oscillations will be more. So, number of oscillations more means there is more tendency of the data being attached to the past values, nothing but the past images, which are already generated by the generator. So, when that influence towards the uh fake images more, then the generator will not uh try to have a adversarial learning. That's why we keep the neutral value, that is 0.5. Also, we need some learning rate in order to update the weights in order to have the gradients. That's why we have the learning rate as 0.0002.

[30:36] So, we define the Adam optimizer for the generator as well as the Adam optimizer for the discriminator. So, now coming to training the generator as well as the discriminator. So, how the training takes place step-by-step on the images. So, we are considering all the images according to the batch size, all the training images. So, we also want the Z, which is the latent vector, which is generated by the noise. So, how do we generate this random noise or a Gaussian noise? It is based on random value generation from a normal distribution based on the batch size 128 and the latent variables 100.

[31:21] So, we have to have the noise, which is of which is having the rows as 128 and columns as 100. So, 100 different dimensions or variables. And then, we make use of the noise in order to generate the images during the training phase. And the discriminator, which makes use of the images during the training phase. So, both are embedded in the tape. Tape means it is used to store some values. So, we have the gradient tape, wherein the updation of the weights takes place based on the loss values, based on the gradient values. Or we can say how much loss is incurring with respect to the weight. That will help us in getting the gradient values.

[32:07] So, we are considering both the generator and the discriminator. Now, what is a fake output that is understood by the discriminator? It is how much ever generated images given by the generator during the train training phase. So, generator loss will be based on the fake output. And the discriminator loss is based on the real output because it considers the real images as well as the fake images. So, there is difference between the generator loss and the discriminator loss. So, generator loss is the set of loss which is dependent on the fake images which is generated by the generator. So, it takes only the Z or the latent vector Z. That's why we have only the fake output.

[32:58] But, the discriminator takes the combination of images. That is the real images as well as the fake images. The real images are the MNIST data set images. The fake images are the images from the generator. Now, how is the gradient calculated? It is based on the generator loss, okay, and the set of trainable parameters or the vari-

ables. Similarly, how is the gradient updation for the discriminator? It is based on the discriminator loss and the trainable variables.

[33:31] So, put together or we have to zip or we have to have a track of all the generator loss that is happening over 20 epochs. Similarly, the discriminator loss which is happening over 20 epochs. So, finally we have to return both the generator loss as well as the discriminator loss. So, that is all about having the optimizers and how to update the gradients or the weights in order to get the new weights based on the old weights.

[34:00] Now, coming to coming to training the DCGAN. So, already we defined the number of epochs as 20 for epoch in the range 20 that is 0 to 19. And for every image in the batch size that is given, we have to calculate the generator loss as well as the discriminator loss on every image batch. And let us print the epochs, the number of epochs as well as the associated generator and the discriminator loss up to four points up to four numbers after the decimal point. That's why we have four numbers displayed after the decimal point.

[34:38] So, as the epochs increases, we are seeing that the generator loss is decreasing. So, it started with a very minimal loss, that is 0.65. Then, the generator loss was 0.84, 0.82. Likewise, it is gradually decreasing. That's because initial adversarial learning, the generator will be very weak, and over the period of time, it will try to learn or generate the fake images by gaining some knowledge from the present images or from the noise.

[35:11] But, in case of discriminator, there is variation in the loss. Initially, it was 1.45, then 1.19, 1.26. So, there is variation in whatever it is able to classify. So, at the end of 20th epoch, we have the generator loss as 0.81. We have the discriminator loss as 1.32. So, both are going hand in hand. So, finally, we will see or we will generate the MNIST data from the random latent vectors or from the random noise.

[35:47] So, let us generate some 16 images based on the noise and the latent dimensions as 100. So, generated images is a variable in which the generator stores the noise generated. Okay? So, we can convert it into a set of numbers, images displayed as number of images displayed as numbers as well. So, generated images + 1 / 2. So, for I in the range 16, we have to display 16 different images. So, four rows, four columns, and at each iteration, at each iteration, we want a placeholder for the uh. That's why we have I plus one. Let the figure size be eight eight {comma} eight eight inches in width and eight inches in height.

[36:36] So, let us show the generated images. So, color map will be equal to gray, and we don't want any label uh for the Y and the X axis. That's why we'll keep it off. And let us have a tight layout. Tight layout means it tries to uh stay within the window. So, showing off of the generated images. So, we can see some sneakers, boots, T-shirts, bag, no shirt. All these are the set of images that are generated. So,

if we try to compare the images generated from the DCGAN when compared to the vanilla GAN, this is a much better uh representation or the construction of the image are of uh upper quality or higher quality in case of DCGANs when compared to the vanilla GANs. Because the DCGANs make use of the

[37:32] convolutional layers, which will try to uh maintain the hierarchy of layers, which will uh take the edges, maybe the texture, the shape, the points, lines, or the object parts. So, it will cover all the low-level features up to the high-level features, which happens during the filtering process. So, by using the low-level uh filters, maybe the low-level kernels up to the high-level kernels, this kind of hierarchy is tried to maintain uh in case of DCGANs.

[38:02] Also, we can see that when we compare the vanilla GAN with a DCGAN, the number of trainable parameters are comparatively less to the architecture or the deep architecture that we have uh built in case of um DCGANs. That's why DCGANs have a upper hand over the basic vanilla architecture, uh which is made up of only the fully connected layers. Also, the drawback in case of vanilla GAN is that the use of fully connected layers which tries to take away the spatial dimension or the spatial structure of the original image. But in case of DC GANs, the spatial structure or the spatial dimensions will still be maintained because we have the option called as padding of zeros over the boundary of the image. So, with this

[38:48] concept of convolution, we are trying to maintain the spatial features. So, with the help of deep convolutional GANs, we have seen that how the convolution layers have a upper hand over the fully connected layers that was used in case of vanilla GAN. But the only drawback in case of deep convolutional GANs is that we do not have the control over generation of the images. That is, the generation of the image is not class specific. Because when we say, "Try to generate some 16 samples." It has randomly generated some accessories like bags, sneakers, boots.

[39:26] Uh but uh also it has generated some clothes. But it is not class specific. So, in case we are more interested towards class specific generation of the data, we will have to move on to the next advanced GAN, that is conditional GAN. So, as the name suggests, it is conditioned over the class. What kind of label we want or what kind of class we want. So, in the next hands-on, we will see how the conditional GAN is better when compared to the DC GANs.

[39:58] Thank you.